

パーソナルコンピュータ研究部 (パー研) の 部誌 2025

パー研の部員たちが、
おのおの書きたいことを書いたものを
まとめました。
興味があるものを読んでみてください。

LINE Messaging APIを使ってみた話

千葉雄飛

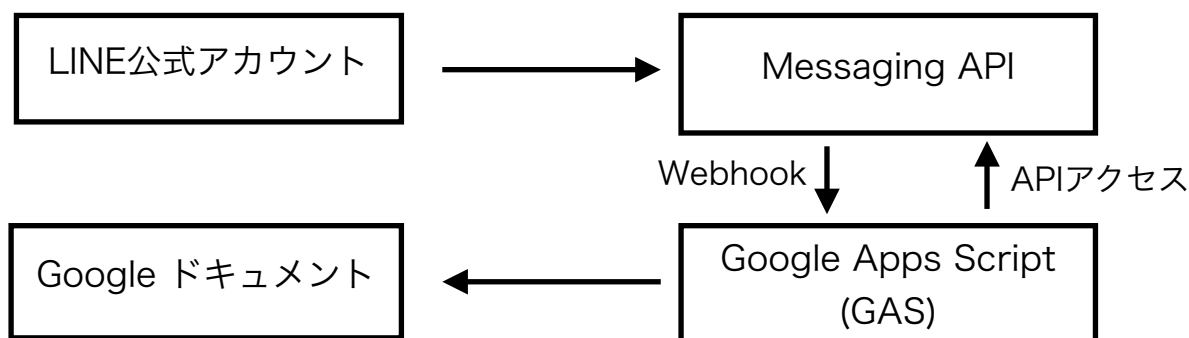
LINEにはチャットボットの管理のために利用できるMessaging APIがある。これを、学校の課題で使ってみたときのことをまとめてみた。

1 背景

学校の英語の課題で「ドキュメンタリー動画を作成する」という課題があった。この課題ではその動画の内容の検討などの過程について記録を残すことが必要であった。これらの検討の多くはLINEグループ上で行われると考えられたため、Messaging APIを利用して、Google ドキュメント上にLINEのログを残せておいたら便利だと考えた。そのため、数日の突貫実装でこのような仕組みを作ってみることとした。

2 仕組みの概要

概ね下図のような構成とした。



以下、この内容についてみていく。

3 メッセージの受信

3.1 Webhook

LINE公式アカウントがメッセージの受信などのイベントを受け取ると、Messaging APIは、事前に指定したURLに対し、所定の形式のHTTP POSTリクエストを飛ばす。これをWebhookという。

メッセージ受信イベントについていうと、どのユーザーからメッセージが送信されたか、メッセージの内容、いつ送信されたかなどの情報がJSON形式で記述されている。これをGASを利用して受け取ることにした。

3.2 GASのウェブアプリ

GASでは、doGetやdoPostという関数を入れておくと、Webアプリとして利用することができる。GETリクエストはdoGet関数で処理し、POSTリクエストはdoPost関数で

処理する。この機能を利用すれば、Webhookの受信とその処理をGAS上で記述することができる。今回のアプリでは、ボット側が発信することはないので、受信処理に徹すれば良い。

Webhookの処理の流れは概ね以下の通り。

1. リクエストボディをJSONとしてパース
2. イベントの配列をfor文で回しながら、
 - 2.1.メッセージ受信イベントのみ処理
 - 2.2.タイムスタンプ、メッセージテキストを読み出す（メッセージがファイルだった場合は、代替テキストを入れておく）
 - 2.3.発信元のuserIdから、表示名をAPIを叩いて受け取る
 - 2.4.そこで、受け取った情報をもとにGASが提供するGoogleドキュメントを操作するAPIを使って、メッセージ内容とメッセージの発信時刻、メッセージの発信者を記録する

4 引かなかったところ

4.1 Webhookの検証ができない

Webhookは、HTTPリクエストとして飛ばされる仕組みであるため、不正なリクエストが入ってくる可能性がある。（今回に関して言えばログを取るだけなので、不正なリクエストが飛んできても、よくわからないメッセージが記録されるだけだが...）

そのため、不正なWebhookでないか署名を検証すべきだとされている。しかし、署名はHTTPリクエストヘッダに記載されているところ、GASのwebアプリケーションからアクセスできるのはクエリ文字列とリクエストボディだけであり、署名を読み取ることができない。しかし、今回はアプリの性質等を踏まえ、このリスクは受容することにした。

4.2 タイムスタンプを人間可読な形に変換できない

JavaScriptのDateの取り回しがなかなかわからなかったせいで少し詰まった。

4.3 ユーザー情報を取得するAPIが個人チャットとグループチャットで異なる

ユーザー名が取得できなかった場合には、Unknownと表示されるようにしていたところ、個人チャットでテストしている時は、ちゃんと自分のアカウント名が表示されたのに、実運用を始めるとUnknownだらけになった。ちゃんとドキュメント読んでみると、アカウント情報取得のために使っていた `/profile/{userId}` というAPIは、その公式アカウントを友だち登録していないと利用できないことをドキュメントから読み落として、本来は `/group/{groupId}/member/{userId}` というAPIを利用しなければならなかった。

4.4 GASのウェブアプリのデプロイの仕組みがよくわからなかった

デプロイは、デプロイ操作をした時点でのソースコードに基づいて、実行されることを理解しておらず、ホットリロードが効くと思っていたために、ソースコードを直したはずなのに同じバグが起きてて訳がわからないと最初なっていた。

4.5 デバッグ出力が出ない

最もめんどくさいのがこれ。`console.log`も`Logger.log`もどこにも出力されない。実行数画面から見られるはずなのに、「この実行に関するログはありません」と言われ、ネットを探してやり方を探したがうまくいかず、どうにもならなかった。デバッグ出力をドキュメント自体に吐き出すというめんどくさいことをしないといけなかった。さらにバグのせいでエラーが起きて、途中で処理が止まると何も出力されず、ログ画面にも何も出ないからエラーが起きたのかどうかもわからない...という状態で開発をしなければならなかった。

NAND でパソコンを作れるのはなんで？

Tamego

概要

マイクラフトでレッドストーン回路を使ってパソコンを作った……みたいなすごい動画ってありますよね。そんなことができちゃうなんて不思議じゃないですか？そこで、今回はその不思議を論理学（数学）の視点から説明します。

1 節

NAND でパソコンを作ってみる

1 | 論理ゲート

論理ゲートとは

論理ゲートとは、受けとった入力に応じて出力を返す回路のことです。レッドストーン回路の **AND 回路**とか **OR 回路**とかがまさにそうです。ここで、いくつか重要な論理ゲートを説明しておきます。

AND 回路

AND 回路は 2 つの入力を取り、その両方がオンなら、オンを出力する回路です。日本語でいう「かつ」と同じような意味を持ちます。

OR 回路

OR 回路は 2 つの入力を取り、そのうちの片方がオンなら、オンを出力する回路です。日本語でいう「または」と同じような意味を持ちます。

NOT 回路

NOT 回路は 1 つの入力を取り、その入力が入オンならオフを、その入力が入オフならオンを出力する回路です。つまり入力の逆が出力になります。日本語でいう「ではない」と同じような意味を持ちます。

まずはこの 3 つを基本回路として考えていきます。

真理値表と回路記号

基本回路がどのような条件で何を出力するのは言葉で説明することができました。しかし、複雑な回路は簡単に説明することができないかもしれません。そこで、どんな回路も正確・厳密に説明することができるように、**真理値表**というものを導入します。真理値表とは、入力の全パターンを列挙し、それぞれに対応する出力を表にしたものです。全てのパターンを網羅できるので、どんなに複雑な回路でも厳密に表すことができるのです。

また、複数の論理ゲートを組み合わせることを考えたときに、「この AND ゲートの出力がああ OR ゲートの入力 1 につながっていて…」と言葉で全て説明するのは大変なので、論理ゲートを図にしてそれぞれを線で繋ぐことで表現することにします。このときの論理ゲートの図を**回路記号**と言います。ここでは本来の回路記号を簡略化したものを使います。

以下に回路記号と真理値表を示しました。特に真理値表が論理ゲートの説明通りになっていることを確認してみてください。

AND 回路



入力 1	入力 2	出力
OFF	OFF	OFF
OFF	ON	OFF
ON	OFF	OFF
ON	ON	ON

OR 回路



入力 1	入力 2	出力
OFF	OFF	OFF
OFF	ON	ON
ON	OFF	ON
ON	ON	ON

NOT 回路



入力	出力
OFF	ON
ON	OFF

2 | 新しく論理ゲートを作る

作るもの

基本回路をこれまでに紹介してきましたが、それ以外の論理ゲートももちろん存在します。ここではその一例として、**XOR 回路**について考えていきます。XOR 回路とは、OR 回路の特殊版のようなものです。基本的に XOR 回路は OR 回路と同じふるまいをしますが、入力の両方ともがオンのときには XOR 回路はオフを出力します。つまり、XOR 回路は 2 つの入力のうちどちらか一方のみがオンのときだけにオンを出力します。以下に回路記号と真理値表を示します。

XOR 回路



入力 1	入力 2	出力
OFF	OFF	OFF
OFF	ON	ON
ON	OFF	ON
ON	ON	OFF

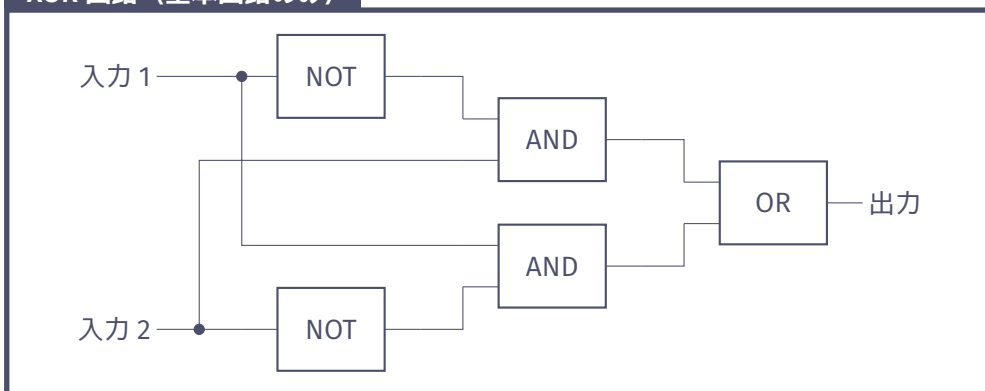
さて、この XOR 回路ですが、実は基本回路のみを使って表すことができます。その方法を少し考えてみてから次に進んでみましょう。

XOR の作り方

XOR のパズルは解けましたか？「2 つの入力のうちどちらか一方のみがオン」という条件は「片方の入力がオンでもう片方の入力がオフ」という条件に言い換えることができるという事実を使うと考えやすいかもしれません。

答えは以下のようになります。

XOR 回路（基本回路のみ）



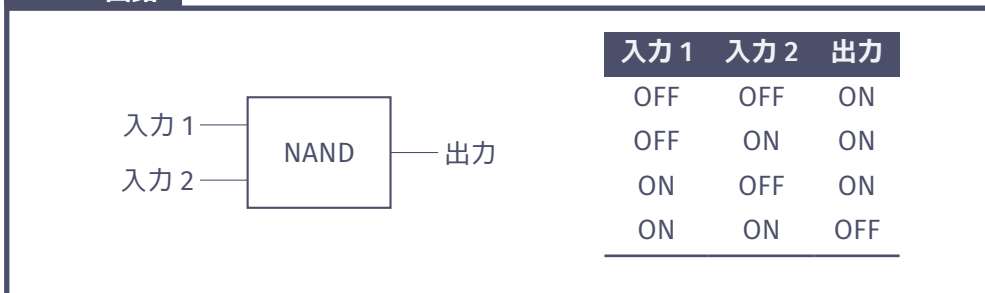
ちょっと複雑ですね。ともかく基本回路のみで XOR 回路を表すことができると分かりました。

3 | NAND で論理ゲートを作る

NAND とは

さて、今回の主役である **NAND 回路** がついに登場します。NAND 回路は AND 回路の出力を逆転させたものです。以下に回路記号と真理値表を示します。

NAND 回路



基本回路と比べて使いにくそうですが、NAND 回路の持つ性質はとても強力なものです。どんな性質を持つのか見て行きましょう。

基本回路を表す

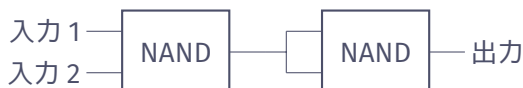
実は NAND 回路のみを使って基本回路を表すことができます。またこれも少し考えてみてから、答えを見てみましょう。まず NOT 回路を表して、AND 回路と OR 回路を NAND 回路と NOT 回路を使って表わしてみると分かりやすいかもしれません。真理値表による考察も有効でしょう。

答えは以下のようになります。

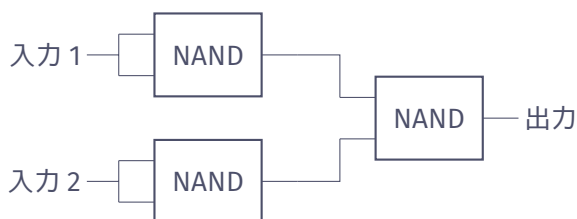
NOT 回路 (NAND のみ)



AND 回路 (NAND のみ)



OR 回路 (NAND のみ)

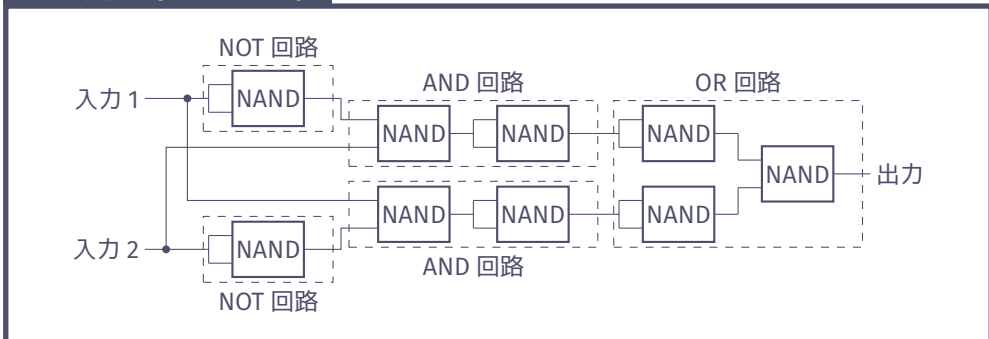


NAND 回路だけで基本回路を作ることができました。すると、基本回路だけで作れる回路は NAND 回路だけで表わせるということが分かります。これが重要な性質の 1 つです。この性質を使って XOR 回路について考えていきましょう。

XOR を表す

XOR 回路は基本回路のみを使って表すことができました。そして、基本回路は NAND 回路のみで表現することが可能でした。したがって、XOR 回路は基本回路のみを使って表すことができます。XOR 回路を基本回路のみで表わし、それぞれの基本回路を NAND 回路のみで表した形に置き換えると、以下のような NAND 回路のみで表された XOR 回路が得られます。

XOR 回路 (NAND のみ)



ちょっとごちゃごちゃしていますね。しかし、たった 4 つの NAND 回路のみで XOR 回路を表すことが実はできます。ここでは解説しませんが、自分で考えてみると面白いかもしれません。

4 | もっと複雑なものを作る

オンオフから数字の世界へ

これまでは論理ゲートについて考えてきましたが、それだけだとなんだかパソコンっぽくはないですね。そこで、足し算をする回路を考えましょう。その前にオンオフで数字を扱う方法を考えましょう。

もし 2 進法を理解しているのなら、半加算器の部分まで読み飛ばしても構いません。また、2 進法の説明をある特定の進数に頼らないでしたので、かえって分かりにくくなってしまっていると思います。わかりづらいな、と感じたら無理せずもっと分かりやすいネット上の文献をあたってから続きを読むことをオススメします。

オンとオフの 2 種類の状態で 0, 1, 2, 3, ... とたくさんの数字を表すのは一見不可能に思えるかもしれません。そこで、今私たちが使っている数字の仕組みを観察して、オンオフの世界に応用できないかを考えてみましょう。

まずは 0 から順番に数を数えていきましょう。

0, 1, 2, 3, 4, 5, 6, 7, 8, 9

この次には何が来るでしょうか。そうですね、10 です。9 が 1 増えると 10 になるという現象は**繰り上がり**として知られているものですね。もっとも、繰り上がりは当たり前と感じられると思いますが、ここで何が起きているかを考えてみましょう。

私たちが普段扱う数字は

0, 1, 2, 3, 4, 5, 6, 7, 8, 9

の 10 個の文字列のみからで構成されます。ここで、1 の次は 2、5 の次は 6、といったように、数を表す文字列同士で順序関係を考えることができます。さて、9 の次が 10 であるというのは次のように考えることができます。

- 9 の次の、数を表す文字列はないから、とりあえず 0 に戻る。
- 「9 の次の数」が 1 つと 0 ということを表すために、0 の左横に 1 を添えて、10 と表記する。

なんだかややこしいかもしれませんが、繰り上がりを言語化してただけです。この説明における「9 の次の数」が何個分あるかを表すところは「10 の位」と呼ばれるものですね。繰り上がりの説明を「1 の位」「10 の位」という言葉を使って次のように書くと分かりやすいかもしれません。

- 9 の次の、数を表す文字列はないので、「1 の位」はこれ以上増やせない。よって、「1 の位」を 0 にリセットする。
- 「1 の位」で「あふれた」分を表すために、「10 の位」を増やして 1 にする。

このようにして位を導入することによって考えやすくなったと思います。

とくに説明もなく位という言葉を使いましたが、ここで簡易的に定義してしましましょう（これは一般的な定義とは異なり、厳密性が失われています）。

位（一般化バージョン）

数を表す文字列を 1 つ持つような構造のこと。位の数とは、その位が持つ文字列が表す数のことを指す。また、位には順序関係がある。

私たちが普段使う数字の表記法において、最初の位は「1 の位」と呼ばれます。その次が「10 の位」で、その次は「100 の位」です。また、最も小さい数というのは、全ての位が、数を表す文字列の中で一番前にある文字列であるときのことを指すものとすると自然である。実際、普段の数字の世界において、最も小さい数は 0 ですね。ひょっとすれば、位の数字の分だけ、その位の「単位」があると考えてのが一般的なのかもしれませんが、しかし、ここでは単に数を保持する構造体として定義しました。それは、位という概念は繰り上がりを説明するための補助的な概念であるためです。その繰り上がりの一般的なルールは以下のようになります。

繰り上がり（一般化バージョン）

ある位 X が、次の 数を表す文字列 を持たないような 数を表す文字列 を持っているとする。このとき、位 X の数を、位 X の数の次の数にするためには、位 X を最初の 数を表す文字列 にして、位 X の次の位 Y の数を、位 Y の数の次の数で表す。

また、ある数字の次の数というのは、最初の位 X の数を、位 X の数の次の数にする操作によって得られるような数字と説明できます。例として、私たちが扱うような数字の世界に限定すれば、この繰り上がりのルールは次のように書けます。

普段の数字の世界における繰り上がり

ある位 X が 9 であったとする。このとき、位 X の数を、位 X の数の次の数にするためには、位 X を 0 にして、位 X の次の位 Y の数を、位 Y の数の次の数で表す。

このようなルールを持つことで、数を表す文字列が 10 個でも、位を用意することによって、たくさん数を文字列の組合せによって表現することができます。同様に、繰り下がりも表すこともできますが、繰り上がりとはほぼ逆の操作で直感的にも理解しやすいはずなので、ここでは省略します。

この一般的なルールを考えたことによって、もしも数を表す文字列が 2 つだけ、すなわちオンとオフだけだったらどのように数を表すことができるかということを考えることができます。ここから先では、表記の簡略化のためにオンを 1、オフを 0 として表記することにします。

オンオフの世界において、数を表す文字列は 0, 1 の 2 つだけです。0 の次が 1 という順序関係を持ちます。このとき、繰り上がりのルールをオンオフの世界に限定した形で表記すると次のようになります。

オンオフの世界の繰り上がり

ある位 X が 1 であったとする。このとき、位 X の数を、位 X の数の次の数にするためには、位 X を 0 にして、位 X の次の位 Y の数を、位 Y の数の次の数で表す。

このルールに基づき、0 をスタートとして、次の数を順に列挙していきます。

0, 1, 10, 11, 100, 101, 110, 111, 1000, ...

このようにして、私たちの世界でいう 8 までの数を書くことができました。この作業はもっと長く続けることができます。私たちの数字の世界において、たくさんの数が表せたのと同様に、オンオフの世界においてもたくさんの数字を表すことができるのです。

ちなみに、私たちの数字の表記法を **10 進法**、オンオフ (0, 1) による数字の表記法を **2 進法**といいます。10 進法と 2 進法は相互に変換することができますし、むしろ 2 進法は 10 進法において定義されることの方が普通ですが、今回はそこには触れないことにします。

2 進法での足し算

さて、数字のお話が続きましたが、この小節の目標は足し算をする回路を作ることでしたね。2 進法の数字の表記法はこれまでで説明してきましたが、足し算などといった演算方法については説明してきませんでした。そこで、足し算に限定して、数字の演算について考えていきます。

数 A と B の足し算の結果は、 A の B 個次の数として得られます (もっと厳密には、 B を B の前の数にして、 A を A の次の数にするという操作を、 B が最も小さい数になるまで繰り返し、終了時の A の数が A と B の足し算の結果であると定義できる)。しかし、これでは大きな数の足し算が大変ですね。そこで、私たちは筆算というツールを持っています。これを一般化してみましょう (証明は興味があれば考えてみてください)。

足し算 (一般化バージョン)

数 A と B の足し算は、最初の位から順に考えていくことで実行できる。

最初の位 X での操作 A の位 X の数を、 B の位 X の数の分だけ次に進めたものにする。ただし、ここでは繰り上がりをすると、位 X の次の位の数を次の進めるのではなく「位 X からの繰り上がり」という数を、その数の次の数にするという操作を代わりに行う。「位 X からの繰り上がり」の数は最も小さい数によって初期化されます。

最初の位以降の位 Y での操作 A の位 Y の数を、 B の位 Y の数の分だけ次に進める。それに加えて、位 Y の 1 つ前の位を X として、「位 X からの繰り上がり」の数の分だけ A の位 Y の数を次に進める。このときの繰り上がりも前述のパターンのときと同様に行う。

実際、10 進法においてこの記述が正しいことは確認できることでしょう (筆算を

ベースに定義したものなので当然といえそうですが)）。つまり、この
ルールの重要なところは、位同士で足し算をすることによって、全体の足し算を考
えることができているということです。

このルールを 2 進法に適応して考えてみましょう。簡単のために、最初の位のみ
で構成される 2 数のみを考えます。そのような数は 0 と 1 のみです。その 2 数の組
合せ 4 通りについて、計算結果を以下に示します。

$$0 + 0 = 0,$$

$$0 + 1 = 1,$$

$$1 + 0 = 1,$$

$$1 + 1 = 10.$$

ここで最後の 1+1 の例にのみ繰り上がりが発生しています。ここでの処理をもっと
細かく見ると、最初の位は 0 になり、「最初の位からの繰り上がり」が 1 となるとい
う操作が行われています。その後、最初の位の次の位について操作が行われて、最
終的に 10 という計算結果となります。他にも例を挙げておくので、どのような処
理が行われているか考えてみましょう。

$$11 + 1 = 100,$$

$$110 + 11 = 1001,$$

$$101 + 11 = 1000.$$

このようにして、2 進法の数における足し算を考えることができます。

半加算器を作る

ようやく導入が終わったので、足し算をする回路を考えていきます。今回作るの
は、最初の位のみで構成される 2 進法の 2 数 A, B の足し算を計算する回路です。つ
まり、足し算のルールにおける「最初の位 X での操作」の部分だけを実装すればよ
いことになります。 A, B は 1 つの位によって表すことができるので、それぞれ入力
1、入力 2 と対応させることができます。そして、与えられた入力をもとに、「位 X か
らの繰り上がり」を表す出力 1、位 X を表す出力 2 をそれぞれ出力できれば、足し
算の「最初の位 X での操作」を表せたことになります。

すると、以下のような真理値表を考えればよいことが分かります。この真理値表

入力 1	入力 2	出力 1	出力 2
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

は以下の計算（再掲）

$$0 + 0 = 0,$$

$$0 + 1 = 1,$$

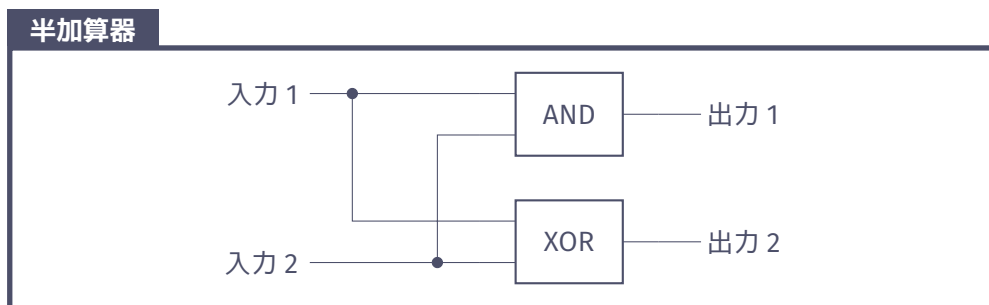
$$1 + 0 = 1,$$

$$1 + 1 = 10$$

と対応していることが分かるでしょう。

さあ、この回路を実装してみましょう。今まで扱ってきた論理ゲートを 1 つずつ使うことでそれぞれの出力を表すことができますよ。

答えは次のようになります。



この半加算器を複数組み合わせることで、足し算の規則の「最初の位以降の位 Y の操作」を表すことができ、足し算を完璧に表すこともできます。興味がある人はぜひやってみてください。

そしてパソコンへ

このようにして、NAND 回路から始めて足し算まで作ることができました。これ以上先に進むとややこしくなりすぎるので、これ以上の解説は今回しません。

この先に進んでいくと、必要な回路をどんどん NAND で表された部品で作っていくことになり、最終的にはパソコンが完成します。たった 1 つの論理ゲートからパソコンが作れるだなんて衝撃的ですよね。このような理論的な話を実際にパズル感覚で試せるのが「Nandgame」という Web ゲームです。そこでは NAND からパソ

コンを作るパズルゲームを遊べます。とくに専門的な知識を必要とせず楽しめるので、ぜひやってみてください。 <https://nandgame.com/>

2 節

NAND でパソコンを作れる理由

1 | 論理ゲートから論理学の世界へ

論理学とは

さて、いよいよ論理学によって NAND の秘密に迫っていきましょう。そもそも**論理学**とは、簡単に言ってしまうと、論証の正しさを形式的に考えるための学問と言えるでしょう。そもそも、何で論理学という数学が、論理ゲートなどといった回路に関係してくるのか、疑問に思うことでしょう。しかし、この 2 つには密接な関係があるのです。まずは、論理ゲートの概念を論理学の世界に持ち込むことを考えましょう。

命題変数と入力

論理学では、真 (1) か偽 (0) のどちらかを取るような変数を考えます。その変数のことを**命題変数**と言います。前節でオンを 1、オフを 0 と解釈したのと同じように、命題変数の真偽と入力のオンオフは対応関係を持つと言えるでしょう。

論理結合子

日本語では、「A または B」とか「A かつ B」とかのように、A と B に「または」や「かつ」を入れて、新たな条件を作ることや、「A ではない」のように、A 単体に「ではない」を付けることによって、新たな条件を作ることができます。この条件を作り出すという操作を少し検討してみましょう。まず、A, B は命題変数の役割をしていると見て問題ないでしょう。区別するためにそれぞれ P, Q としますね。そして、「A または B」「A かつ B」といった条件に使われている「または」「かつ」を $\vee, \wedge$ という記号でそれぞれ表すと、「A または B」「A かつ B」をそれぞれ $P \vee Q, P \wedge Q$ と表すことができそうです。また「A じゃない」の「じゃない」は $\neg$ で表すと、「A じゃない」は $\neg A$ と書くことができそうです。ここで紹介した $\vee, \wedge, \neg$ のような、新しい条件を作り出す記号は**論理結合子**と呼ばれています。まずはこの 3 つを基本として考えていきましょう。

論理式の文法

結合子の使い方がちゃんと明示されていないので、ここでしっかり定義しておきましょう。論理学では、命題変数を結合子によって 1 つ以上組み合わせた構造を**論理式**と言います。ついさっき「条件」などと呼んだものです。論理式の文法は次のように定義されます。

論理式の文法

- 命題変数そのものは論理式である。
- A, B を論理式としたときに、 $(A \wedge B)$, $(A \vee B)$, $(\neg A)$ はそれぞれ論理式である。
- 以上によって論理式とされるものだけが論理式である。

これによって論理式の構造、そして結合子の使い方をハッキリさせることができました。

結合子と論理ゲート

AND と OR $\wedge, \vee$ はどちらも、2 つの論理式を取って、1 つの論理式を表すという構造を持っている。この点で、入力を 2 つ取って出力するという AND と OR の論理ゲートと構造上対応している。

NOT $\neg$ は 1 つの論理式を取って、1 つの論理式を表すという構造を持っている。これは、入力を 1 つ取って出力するという NOT の論理ゲートと構造上対応している。

といった風に、結合子と論理ゲートには構造的な対応関係が見られる。

2 | 結合子に意味を与える

真理値表による意味付け

ここまでで、論理式がどのような構造を持つのかを考えてきた。しかし、結合子の意味はまだ定義されていない。ここで真理値表によって、各結合子に意味を与える。

結合子の意味

P	$\neg P$
0	1
1	0

P	Q	$P \wedge Q$	$P \vee Q$
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	1

ここで注意したいのが、与えられた真理値表は、論理ゲートの定義に使われたものと同一のものであるということである。つまり、前小節において、結合子が論理ゲートと構造上対応するという関係が分かり、意味的な対応関係を持つことが分かったので、結合子と論理ゲートは構造上・意味上で同一である。したがって、その2つの概念を同一視することができる。

3 | 十全性

十全とは

ある特定のいくつか結合子のみを使った論理式によってどんな真理値表をも表現できるとき、その特定のいくつかの結合子は**十全**であると言う（厳密には、表現するのは真理値表自体ではなく真理関数の方）。つまり、ある結合子の集合が十全であると分かったら、その結合子さえあればどんな複雑な真理値表でも表現できるということです。また、結合子と論理ゲートの対応関係から、十全であると分かった結合子の集合に対応する論理ゲートの集合を使えば、どんなに真理値表が複雑であっても、すべての回路を表すことができるということです。ここでのゴールは NAND が十全であることを示すことです。NAND が十全であるのならば、NAND 回路を使えば（真理値表で表現できる）どんな回路も作れるということで、これはパソコンが NAND 回路だけで作れるということを理論的にサポートする。その前に、比較的示しやすい「AND, OR, NOT は十全」ということを示す。

AND, OR, NOT は十全

実は以下のアルゴリズムに従えば、任意の真理値表を表すことができる。

- 論理式の集合を S とする。
- 真理値表を1行ずつ見ていく。
- その行の出力が1であるか？

はい – 論理式の集合を T とする。

- 各命題変数について、0 が割り当てられているときはその命題変数の否定を、1 が割り当てられているときはその命題変数そのままを、 T に追加する。
- 全ての命題変数についてその操作が完了したら、 T の中身をすべて取り出して、 $\wedge$ で繋げる（命題変数がある行の割り当てと全く同じ割り当てのときに限って 1 になる論理式である）。その論理式を S に追加する。

いいえ 何もしない。

- 全ての行について操作が完了したとき、 S は空であるか？

はい 決して真になることのない論理式を $\wedge, \vee, \neg$ のみを使って表せば、真理値表をその論理式で表現できる。。

いいえ S の中身を全て取り出して、 $\vee$ で繋げる（どれか一つでも真であれば、真となるような論理式となる）。その論理式によって、真理値表を表すことができる。

ためしに、この手順を XOR 回路を例に挙げて追ってみましょう。XOR 回路の真理値表は次の通りです。

XOR の真理値表		
P	Q	$P \text{ XOR } Q$
0	0	0
0	1	1
1	0	1
1	1	0

- 集合 S を考える。
- 1 行目の出力は 0 なので関係ない。
- 2 行目の出力は 1 である、
 - 集合 T を考える。
 - P の割り当ては 0 だから、 $\neg$ を頭に付けた $\neg P$ を T に追加する。
 - Q の割り当ては 1 だから、そのままの Q を T に追加する。
 - 全ての命題変数をチェックしたので、 T の中身を取り出して、 $\wedge$ で繋げた論理式、すなわち $\neg P \wedge Q$ を S に追加する。
- 3 行目の出力は 1 なので、2 行目と同様にして、 $P \wedge \neg Q$ を S に追加する。
- 4 行目の出力は 0 なので関係ない。

- 最後に、 S の中身を取り出して、 $\vee$ で繋げる。すると論理式 $(\neg P \wedge Q) \vee (P \wedge \neg Q)$ によって与えられた真理値表を表現できる。

この手法は命題変数が2つの真理値表でなくても使えるようになっている。

NAND は十全

さて、ようやく NAND が出てきます。NAND の結合子は $|$ で、真理値表は次の通りです。

NAND の真理値表

P	Q	$P Q$
0	0	1
0	1	1
1	0	1
1	1	0

ここで、前節において、AND 回路、OR 回路、NOT 回路を NAND 回路で表したことを利用する。すると、 $\neg P$ は $P|P$ と書けるし、 $P \wedge Q$ は $(P|Q)|(P|Q)$ と、 $P \vee Q$ は $(P|P)|(Q|Q)$ と書けるので、AND, OR, NOT の結合子は NAND のみで表すことができる。ここで、任意の論理式は AND, OR, NOT を使って表すことができるので、それらを NAND に置き換えることによって、任意の論理式が NAND で表されることになる。したがって、NAND は十全である。

ついに NAND が十全であることを示すことができました！

3 節

おわりに

長かったですね…論理ゲートから始まって、2 進法を導入し、そして最後には論理学まで踏み込みました。この長かった議論を踏まえて、最後にタイトル「NAND でパソコンを作れるのはなんで？」の答え合わせをしましょう。その答えは「NAND は十全だから」でした。

機械学習を用いた強力なオセロ用評価関数の作成

高 2-1 笹倉颯人

1 評価関数とは

皆さんはオセロや将棋、チェスといったボードゲームで遊んだことはありますか？友達と、家族とやったことがあるという人が多いと思いますが、コンピューターと対戦したことはありますか？僕は自分の iPad で将棋でコンピューターと対戦して遊ぶことがよくありました。さて、そんなボードゲームの「コンピューター」ですが、どうやって差し手の良し悪しを判断しているのでしょうか？最近であれば、まず「機械学習」や「ディープラーニング」といったことを思いつくかもしれません。コンピューターに学習させて、それをもとにどこに置くべきかを予測する、という方法もあります。しかし、チェスや将棋など、片方がある程度有利な時、相手はそれと同じだけ不利なゲーム（二人零和有限確定完全情報ゲーム）では、それとは別にタイトルにもある「評価関数」によって、ある盤面があった時、それがどれくらい有利かどうかを数値化し、その数値によって判断することができます。

2 簡単な評価関数

では、評価関数が何かについての説明をしたところで、オセロ向けの簡単な評価関数を考えてみましょう。

	a	b	c	d	e	f	g	h
1	30	-12	0	-1	-1	0	-12	30
2	-12	-15	-3	-3	-3	-3	-15	-12
3	0	-3	0	-1	-1	0	-3	0
4	-1	-3	-1	-1	-1	-1	-3	-1
5	-1	-3	-1	-1	-1	-1	-3	-1
6	0	-3	0	-1	-1	0	-3	0
7	-12	-15	-3	-3	-3	-3	-15	-12
8	30	-12	0	-1	-1	0	-12	30

※ 1

この図を見てください。隅は値が大きく、その隣は負の値など、初心者も含め多くの人が知っているオセロの最も基本的な特徴が組み込まれています。中心のますの評価値が高く、その周りが小さいことから、「相手に囲ませる」といった基本的な戦略も読み取れます。

3 機械学習を利用した評価関数

しかし、先ほど紹介したものには大きな問題があります。例えば、角を取っていたとしても、その周りのマスがとられていては、あまりいい状態とは言えません。その問題を解決するために、マス一つ一つではなく、いくつかのマスを含わせてその形によって盤面を評価します。一般的にこれはパターン評価と言われています。



これは強力なオセロ AI として有名な Edax が用いているパターンの一部です。このように、40 個ほどのマスを合わせて評価することで、不利・有利な形を見つけ、その和を評価値とします。では、その各パターンの評価値はどのようにして求めるのでしょうか？例えば、10 マスで 1 パターンとしている場合、そのパターンには石の配置が 3^{10} 通りあります。これは人力で計算・代入できる数ではありません。ここで使えるのが機械学習です。オセロの棋譜をもとに、どの形があったときにどのくらいの石差で終局したのかというデータを大量に用意し、それをもとに誤差が最小になるように値を求めていきます。

ここで、 N 個の局面データ p_i とその局面に対する真の評価値 x_i 、そして評価関数が出力する評価値を $f(p_i)$ とします。すると、各パターンの重み w_j の更新式は、

$$w_j = w_j + 2\alpha \sum_{i=1}^N (x_i - f(p_i))$$

ここで α は学習率です。学習率というのは、1 度にどのくらい重みを更新するか決めるものです。例えば、誤差の合計

$$\sum_{i=1}^N (x_i - f(p_i))$$

が 300 だったとします。これをそのまま重みの更新量としてしまうと、当たり前ですが最適解を通り過ぎてしまいます。そのため、少しずつ買いに近づけていくために $\alpha \in [0, 1]$ をとってあげます。誤差に α をかけた値を各パターンの重みに足していくことで、少しずつ誤差を小さくしていきます。誤差を 0 にすることは基本的にできませんが、最も小さくなるようにします。このような手法を勾配降下法、最急降下法 (Gradient Descent) などと言います。(上の式はわかりやすくしたもので、正確なものではありません) 最急降下法では全データに対する誤差の平均値を取りますが、そ

れに対してミニバッチ勾配降下法と呼ばれるものは、データのうちあらかじめ決めておいた分をランダムに選び、そのうちの平均を取って誤差とし、それを全てのデータに対して繰り返します。後者は局所最適解への収束をある程度防ぎ、計算も軽いというメリットがありますが、オセロ AI の評価関数に関しては全てのパラーンにおける評価値の初期値を 0 としておけばあまり問題がなく、正確な勾配が計算できる通常のフルバッチ勾配降下法が使われるようです。こちらは CPU で処理をすると、データ量が多い時にパラメータを 1 回更新するのに数 10 分 1 時間ほどかかってしまうことがあるので、誤差の計算には GPU を使います。GPU であれば、大量のデータそれぞれにスレッドを割り当てることができ、CPU の 1000 倍ほど高速に計算できます。実際に、僕の環境では Apple M1 Pro CPU で 20 分かかっていた処理が、NVIDIA GeForce RTX 3060 では 1 秒前後しかかかりませんでした。

こうすることで、各パターンについて重みをつけることができます。棋譜に出てこなかった局面は評価値を 0 とせざるを得ませんが、大量に寄付を用意していれば、基本的に出てこなかったパターンは無視してもいいことが多いようです。データが少ない場合は、深層学習によって評価値を予測させられるモデルを作ることでもあります。

4 終わりに

機械学習を用いたボードゲーム AI の評価関数 (主にオセロ) について簡単に解説しましたが、どうだったでしょうか？もし、ボードゲーム AI を作ろうと考えている人がいて、役に立ったなら嬉しいです。

参考文献

- [1] オセロの作り方 <https://uguisu.skr.jp/othello/5-1.html>
- [2] もしオセロの初心者が Edax の評価関数を読んだら <https://qiita.com/tanaka-a/items/6d6725d5866ebe85fb0b>

電脳部誌2 『Tamashi』 のジャケット制作について

ぷんち

アルバムの顔となるジャケットはそのCDを最初に印象付ける非常に重要なものです。この記事では僕がどのようにアルバムのジャケットを制作したかを振り返り解説したいと思います。

1.コンセプト決め

『Tamashi』を制作するにあたり、2年前ぐらいから虹色の炎をジャケットにしたいな〜とぼんやり思っていました。(魂のイメージ。)

以下参考にした資料を載せます。



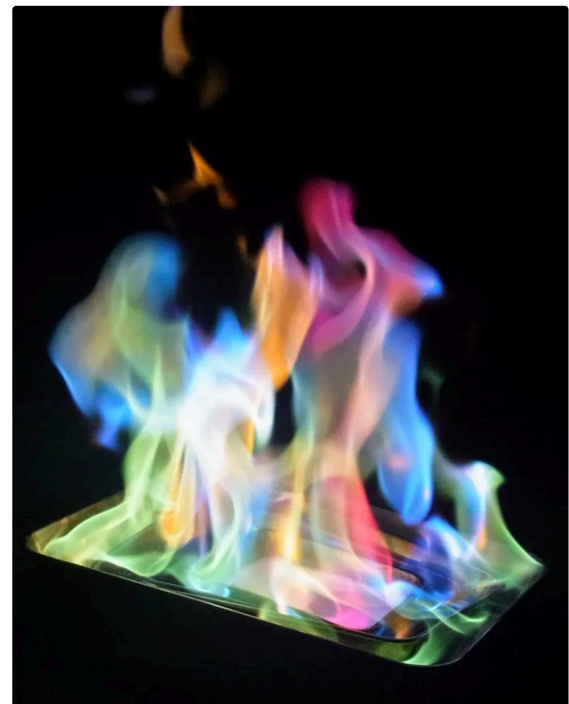
Zedd - 『True Colors』 ど真ん中に置く構図など参考にしました。名盤。



Bonbero - 『Karenai』 最初は背景を白ではなく黒にしようと考えていました。



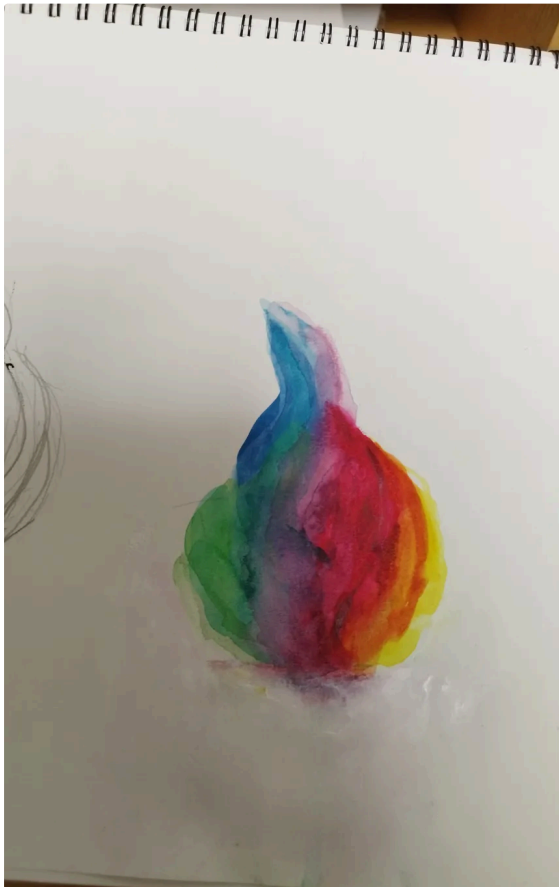
『地縛少年花子くん』より、白杖代



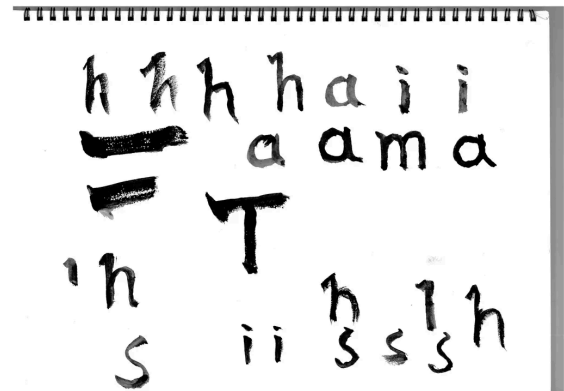
炎色反応。炎は意外と複雑な形をしている。炎を観察するうちに流れを意識して描くといいかもな〜と気づいた。

2.アートワーク制作

Illustratorの使い方もわからないしカッコいい幾何学デザインも作れる自信がない、なら自分で描いてしまおう!と水彩絵の具を買いに行きました。結果的に色が混ざり合っていることを表現するのに油絵やデジタルではなく水彩画を選んだのは良い選択だったな〜と思います。(部誌では白黒になってしまっていると思いますが...) 水彩画の経験は一切なかったので、Watercolor by Shibasakiチャンネル様の動画を大いに参考にさせていただき、グラデーションなどの技法を見様見真似でやっていました。柴崎先生はやっぱりすごいんだなと実感しました。



スケッチブックに描いたアートワークの原画。



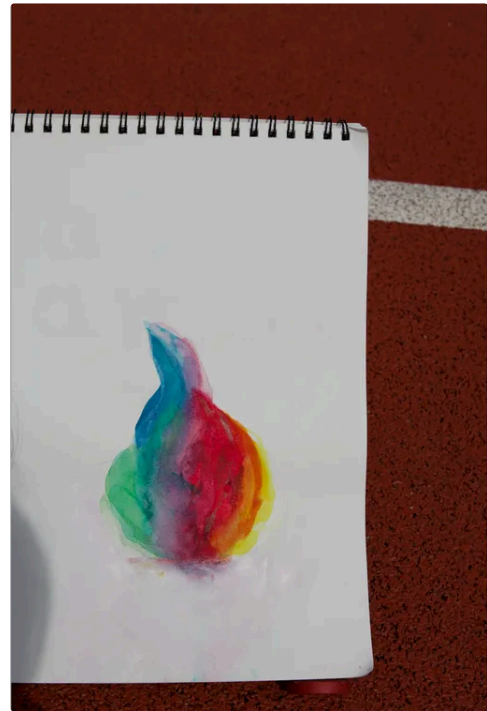
“Tamashi”という文字は筆でスケッチブックに書きました。字が下手なので保険に一文
字ずつ書き並べています。怖いね。

3.画像編集

素材を全て揃えた後は、まず写真部の友人にいいカメラを使って原画を撮ってもらいました。そしてその素材をPhotoshopに取り込み、体裁を整えました。何気に曲目を書いてある裏面を作るのに、どのフォントを使えば良いかとかマージンとかバランスとかを調整するのが大変だった覚えがあります。



Photoshopの作業画面。キャンバス？和紙？のような背景はPhotoshopのAI生成で出力しました。便利。



写真部の友人に撮っていただいた写真。感謝。

4.まとめ

絵を描いたり、初めてPhotoshopに触ってみたり、色々慣れないことが多かったですがとても新鮮な体験で、結果的にアルバムにあった満足いくジャケットができたと思います。お読み頂きありがとうございました。

電脳部誌3 『Deities』 を解剖する

ぷんち

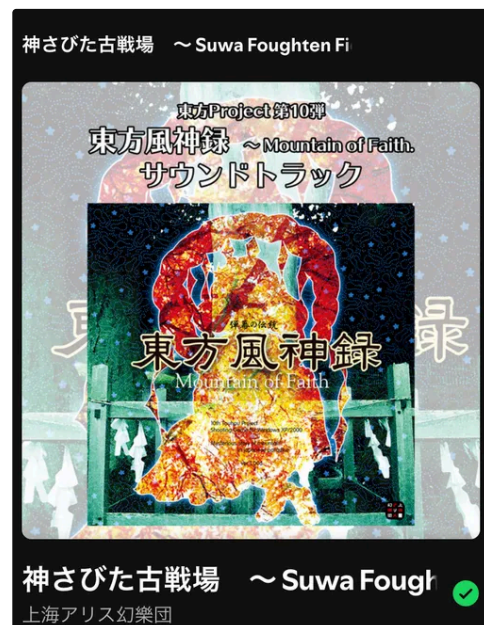
この記事ではオリジナル自主制作CD、『Tamashi』に収録されている一曲、『Deities』を解剖して一つ一つ解説してみようと思います。

着想

一作前、ダークな曲調の『Voyage』という曲を完成させた後、前の曲とは違った明るい曲調で踊りたくなるような曲を作りたいと思い、制作を始めました。そこで、自分が主に作っていたProgressive Houseというジャンルに、僕が好きな、東方Projectの曲や、Xomuさん、Kirara Magicさんを筆頭としたOriental Houseというジャンルから着想を受け、和風で盛り上がるダンスチューンを作ろうというところからスタートしました。



Xomu & Justin Klyvis - Setsuna (Kirara Magic Remix) 琴の音がベースドロップに非常にマッチしたとてもいい和風EDM曲です。



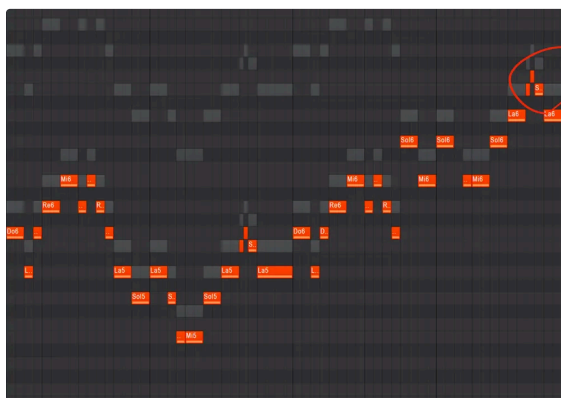
ZUN - 神さびた古戦場 ~ Suwa Foughten Field~ 和の音が他の曲よりも色濃く全面に押し出されていてとても好きです。全面東方のOSTは風神録が一番好き。

ドロップ

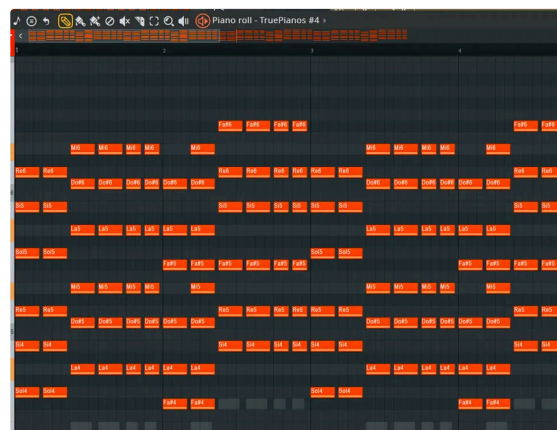
・ドロップとはEDMで最も盛り上がる部分、つまりはサビのことです。和風なドロップを作るにあたり、和風でキャッチーな音階と言われる、ヨナ抜き音階を使いメインメロディーを作りました。また、これはなぜだか僕もまだ良く分かっていないのですが、トリル（32分音符を使い音程を上下させる）を用いると和のテイストが良く出ると感じ、メロディ内でトリルを多用しています。おそらくトリルの、ビブラートやこぶしと似た響きがそれを可能にしているのかもしれませんが。そしてできたメロディーに4-5-3-6進行（いわゆる王道進行）を合わせ、曲の核が完成しました。コードのリズムを刻んでいるうちに物足りなく感じ、上物を追加して進行は最終的にG-A-F#m-F#m7-Bmになりました。

・最初は八長調(C major)で作っていたのですが、移調して試してみたら二長調(D major)がピッタリハマったのでそれで行こう！となりました。

・ドラムにはそこまで重くないキックとスネアを使い、ドロップを爽やかに聴ける仕上がりになったと思います。キックは元々UK Hardcoreというジャンルのサンプルでしたが、その爽やかな音が好みでよく使っています。また、裏打ちのリズムのホワイトノイズなどを入れています。ハイハットやホワイトノイズはグルーブを出す上で非常に重要です。ドラム隊はEDMの心臓だと思います。フィルに拍子木の音を入れました、和風っていいですね。



LeadのメロディーのMIDI。Cメジャーに移調するとわかりやすいが、ペントニックスケール、いわゆるヨナ抜き音階になっている。これがキャッチーかつ和風な音を奏でている。赤丸で囲んだ部分がトリル。



コードのMIDI。(16分音符が) 333322の順番で並ぶ刻み。

編曲と音作り

・何か日本という世界を一発で理解させるようなわかりやすく和のエッセンスを持った音を使いたいなと思い、色々探した結果、拍子木（曲中で使われている木のキンキン！という音）に辿り着きました。これが非常にダンスミュージックと合うのです。ドロップでフィルとして使うもよし、イントロで使うもよし（最初の、カンカンカンカンカン...という部分）、Extended Mixでは裏打ちのハイハットのようないい使い方もしました。実はフリー素材をリバーブとコンプをかけてそのまま使っています。

・イントロで使われている部分は笛と琵琶が合奏をしているような音を目指しました。実はこの琵琶に聞こえる音は琵琶の音源を使っているわけではなく、Kontaktプリセットにあったタンブールという楽器の音を使っています。中央アジアの民族楽器らしいです。意外と違和感ないでしょう？笛はNexusライブラリにあったパンフルートの音を使っています。とてもエスニックな風情が感じられてお気に入りです。

└・ドロップのLead（メインメロディー）の音はピアノとブラック系の音、そして甲高いノコギリ波系の音をレイヤーして作っています。完全オリジナル！とは言いませんが、あまり普段聴いているEDMでは聴かないユニークで爽やかな音作りができたのではないかなと思っています。ブラックの音がコロコロとしたリズムを奏でていて結構好きです。

裏話

・ドロップをある程度完成させた後、これはいい曲になるなと思うと同時にイントロをどう展開するか丸一日悩んでいました。そして気分転換にと夜十時に近所の神社にまで散歩し境内で鼻歌をフンフン歌っていたら、なんとイントロのメロディが降ってきたのです！ルンルンで帰り、スマホでレコーダーを回していたのでそのメロディをMIDIに起こして清書しました。そうやってAメロとBメロが完成しました。なので僕はこの曲にはガチで神が宿っていると信じてます。

・曲が完成した後、タイトルをどうしようかと考えた時、ネットの難読英単語の一覧でたまたま発見した「神々」という意味を持つ"Deities"（発音はディエティーズ /diː.ɪ.tiz/と読まれるそうです。デイティーズではないらしい。）という単語がとてもこの曲にマッチしているなと思い、そのまま曲名にしました。Godsとは違い、Deitiesは多神教（日本の神道とか古代エジプトの宗教）における神様のことを指すそうです。

・この曲のシングルジャケットはそのメロディを思いついた神社の入り口です。神々しい。

・とても気に入っている曲で僕がDJをする"DJ展"にてこの曲を流すつもりです。ぜひ来てね。



神社の入り口。

FL Studioにおいて、**OTT**ではなく**SoundGoodizer**を使うメリット

ぷんち

ない